



Red Hat OpenShift AI Self-Managed 2.25

Release notes

Features, enhancements, resolved issues, and known issues associated with this release

Red Hat OpenShift AI Self-Managed 2.25 Release notes

Features, enhancements, resolved issues, and known issues associated with this release

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

These release notes provide an overview of new features, enhancements, resolved issues, and known issues in version 2.25.3 of Red Hat OpenShift AI.

Table of Contents

CHAPTER 1. OVERVIEW OF OPENSIFT AI	3
CHAPTER 2. NEW FEATURES AND ENHANCEMENTS	4
2.1. ENHANCEMENTS	4
2.2. NEW FEATURES	4
2.3. ENHANCEMENTS	5
CHAPTER 3. TECHNOLOGY PREVIEW FEATURES	7
CHAPTER 4. DEVELOPER PREVIEW FEATURES	11
CHAPTER 5. SUPPORT REMOVALS	12
5.1. DEPRECATED	12
5.1.1. Caikit Standalone ServingRuntime for KServe	12
5.1.2. Deprecated Kubeflow Training operator v1	12
5.1.3. Deprecated TrustyAI service CRD v1alpha1	12
5.1.4. Deprecated KServe Serverless deployment mode	12
5.1.5. Deprecated LAB-tuning	12
5.1.6. Deprecated embedded Kueue component	12
5.1.7. Deprecated CodeFlare Operator	13
5.1.8. Deprecated model registry API v1alpha1	13
5.1.9. Multi-model serving platform (ModelMesh)	13
5.1.10. Deprecated Text Generation Inference Server (TGIS)	13
5.1.11. Deprecated accelerator profiles	13
5.1.12. Deprecated OpenVINO Model Server (OVMS) plugin	13
5.1.13. OpenShift AI dashboard user management moved from ODHDashboardConfig to Auth resource	13
5.1.14. Deprecated cluster configuration parameters	14
5.2. REMOVED FUNCTIONALITY	14
5.2.1. Microsoft SQL Server command-line tool removal	15
5.2.2. Model registry ML Metadata (MLMD) server removal	15
5.2.3. Embedded subscription channel not used in some versions	15
5.2.4. Anaconda removal	15
5.2.5. Pipeline logs for Python scripts running in Elyra pipelines are no longer stored in S3	16
5.2.6. Beta subscription channel no longer used	16
5.2.7. HabanaAI workbench image removal	16
CHAPTER 6. RESOLVED ISSUES	17
6.1. SECURITY UPDATES IN RED HAT OPENSIFT AI 2.25.3 (FEB 2026)	17
6.2. ISSUES RESOLVED IN RED HAT OPENSIFT AI 2.25.2 (FEBRUARY 2026)	17
6.3. SECURITY UPDATES IN RED HAT OPENSIFT AI 2.25.1 (DECEMBER 2025)	17
CHAPTER 7. KNOWN ISSUES	19
CHAPTER 8. PRODUCT FEATURES	34

CHAPTER 1. OVERVIEW OF OPENSIFT AI

Red Hat OpenShift AI is a platform for data scientists and developers of artificial intelligence and machine learning (AI/ML) applications.

OpenShift AI provides an environment to develop, train, serve, test, and monitor AI/ML models and applications on-premise or in the cloud.

For data scientists, OpenShift AI includes Jupyter and a collection of default workbench images optimized with the tools and libraries required for model development, and the TensorFlow and PyTorch frameworks. Deploy and host your models, integrate models into external applications, and export models to host them in any hybrid cloud environment. You can enhance your data science projects on OpenShift AI by building portable machine learning (ML) workflows with data science pipelines, using Docker containers. You can also accelerate your data science experiments through the use of graphics processing units (GPUs) and Intel Gaudi AI accelerators.

For administrators, OpenShift AI enables data science workloads in an existing Red Hat OpenShift or ROSA environment. Manage users with your existing OpenShift identity provider, and manage the resources available to workbenches to ensure data scientists have what they require to create, train, and host models. Use accelerators to reduce costs and allow your data scientists to enhance the performance of their end-to-end data science workflows using graphics processing units (GPUs) and Intel Gaudi AI accelerators.

OpenShift AI has two deployment options:

- **Self-managed software** that you can install on-premise or in the cloud. You can install OpenShift AI Self-Managed in a self-managed environment such as OpenShift Container Platform, or in Red Hat-managed cloud environments such as Red Hat OpenShift Dedicated (with a Customer Cloud Subscription for AWS or GCP), Red Hat OpenShift Service on Amazon Web Services (ROSA classic or ROSA HCP), or Microsoft Azure Red Hat OpenShift.
- A **managed cloud service**, installed as an add-on in Red Hat OpenShift Dedicated (with a Customer Cloud Subscription for AWS or GCP) or in Red Hat OpenShift Service on Amazon Web Services (ROSA classic).
For information about OpenShift AI Cloud Service, see [Product Documentation for Red Hat OpenShift AI](#).

For information about OpenShift AI supported software platforms, components, and dependencies, see the [Red Hat OpenShift AI: Supported Configurations](#) Knowledgebase article.

For a detailed view of the 2.25 release lifecycle, including the full support phase window, see the [Red Hat OpenShift AI Self-Managed Life Cycle](#) Knowledgebase article.

CHAPTER 2. NEW FEATURES AND ENHANCEMENTS

This section describes enhancements in Red Hat OpenShift AI 2.25.1.

2.1. ENHANCEMENTS

Upgrade vLLM Gaudi SW 1.22.2 + vLLM v0.9.0.1 in RHOAI_2.25.1

Support for vLLM Inference Server with Intel Gaudi 1.21 accelerators is now available in Red Hat OpenShift AI 2.25.1.

2.2. NEW FEATURES

Feature Store includes a web-based UI

The Feature Store component now includes a web-based user interface (UI). You can use this UI to view all registered Feature Store objects, including features, data sources, entities, and feature services, as well as their relationships with each other. To enable the UI, you must edit your FeatureStore custom resource (CR) instance. After you save the change, the Feature Store Operator starts the UI container and creates an OpenShift route to access it. For more information about enabling the Feature Store user interface, see [Setting up the Feature Store user interface for initial use](#).

Model registry and model catalog general availability

OpenShift AI model registry and model catalog are now available as general availability (GA) features.

A model registry acts as a central repository for administrators and data scientists to register, version, and manage the lifecycle of AI models before configuring them for deployment. A model registry is a key component for AI model governance.

The model catalog provides a curated library where data scientists and AI engineers can discover and evaluate the available generative AI models to find the best fit for their use cases.

LLM Compressor library added to OpenShift AI workbench images and pipelines

The LLM Compressor library is now generally available and fully integrated into standard OpenShift AI workbench images and pipelines.

This library provides a supported, integrated method to optimize large language models for improved inference, particularly for deployment on vLLM, without leaving your OpenShift AI environment. You can run model compression as an interactive notebook task or as a batch job in a pipeline, which significantly reduces the hardware costs and improves the inference speeds of their generative AI workloads.

Use an existing Argo Workflows instance with pipelines

You can now configure OpenShift AI to use an existing Argo Workflows instance instead of the one included with Data Science Pipelines. This feature supports users who maintain their own Argo Workflows environments and simplifies adoption of pipelines on clusters where Argo Workflows is already deployed.

A new global configuration option disables deployment of the embedded Argo WorkflowControllers, allowing clusters that already use Argo Workflows to integrate with pipelines without conflicts. Cluster administrators can choose whether to deploy the embedded controllers or use their own Argo instance and manage both lifecycles independently. For more information, see [Configuring pipelines with your own Argo Workflows instance](#).

Support added for workbench images

You can now install and upgrade Python 3.12 workbench images in OpenShift AI for your JupyterLab and code-server IDEs.

2.3. ENHANCEMENTS

Support for customizing OAuth proxy sidecar resource allocation

You can now customize the CPU and memory requests and limits for the OAuth proxy sidecar in workbench pods. To do this, add one or more of the following annotations to the notebooks custom resource (CR):

- **notebooks.opendatahub.io/auth-sidecar-cpu-request**
- **notebooks.opendatahub.io/auth-sidecar-memory-request**
- **notebooks.opendatahub.io/auth-sidecar-cpu-limit**
- **notebooks.opendatahub.io/auth-sidecar-memory-limit**

If you do not specify these annotations, the sidecar uses the default values of 100m CPU and 64Mi memory to maintain backward compatibility. After you add or modify the annotations, you must restart the workbench for the new resource allocations to take effect.

The annotation values must follow the Kubernetes resource unit convention. For more information, see [Resource units in Kubernetes](#).

Enhanced workbench authentication

Workbench authentication is now smoother in OpenShift AI. When you create a new workbench, a reconciler automatically generates the required **OAuthClient**, removing the need to manually grant permissions to the **oauth-proxy** container.

Support for flexible storage class management

With this release, administrators can now choose any supported access mode for a storage class when adding cluster storage to a project or workbench in OpenShift AI. This enhancement removes deployment issues caused by unsupported storage classes or incorrect access mode assumptions.

Support for deployment on the Grace Hopper Arm platform

OpenShift AI can now be deployed on the Grace Hopper Arm platform. This enhancement expands hardware compatibility beyond x86 architectures, enabling you to deploy and run workloads on Arm-based NVIDIA Grace Hopper systems. These systems provide a scalable, power-efficient, and high-performance environment for AI and machine-learning workloads.



NOTE

The following components and image variants are currently unavailable:

- The **pytorch** and **pytorch+llmcompressor** workbench and pipeline runtime images
- CUDA-accelerated Kubeflow training images
- The **fms-hf-tuning** image

Define and manage pipelines with Kubernetes API

You can now define and manage data science pipelines and pipeline versions by using the Kubernetes API, which stores them as custom resources in the cluster instead of the internal database. This enhancement makes it easier to use OpenShift GitOps (Argo CD) or similar tools to manage pipelines, while still allowing you to manage them through the OpenShift AI user interface, API, and **kfp** SDK.

This option, enabled by default, is configurable with the **Store pipeline definitions in Kubernetes** checkbox when you create or edit a pipeline server. OpenShift AI administrators and project owners can also configure this option by setting the **spec.apiServer.pipelineStore** field to **kubernetes** or **database** in the **DataSciencePipelinesApplication** (DSPA) custom resource. For more information, see [Defining a pipeline by using the Kubernetes API](#).

Support added for configuring TrustyAI global settings with the DataScienceCluster (DSC) resource

Administrators can now declaratively manage settings such as LMEval's **allowOnline** and **allowCodeExecution** through the DSC interface, with changes automatically propagated to the TrustyAI operator. This unifies TrustyAI configuration with other OpenShift AI components and removes the need for manual ConfigMap edits or Operator restarts.

Support added to move unwanted files to trash directory

You can now increase your container storage by moving and permanently deleting your unwanted files to a trash directory in the Jupyter Notebook. To delete these files, click the **Move to Trash** icon on your Jupyter notebook toolbar and browse through your trash directory. Select the files that you would like to permanently delete, and delete them to prevent full notebook storage.

Updated workbench images

A new set of workbench images is now available. These pre-built workbench images and upgraded packages include Python libraries and frameworks for data analysis and exploration, as well as CUDA and ROCm packages for accelerating compute-intensive tasks. Additionally, they feature runtimes and updated IDEs for RStudio and code-server.

CHAPTER 3. TECHNOLOGY PREVIEW FEATURES



IMPORTANT

This section describes Technology Preview features in Red Hat OpenShift AI 2.25. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

IBM Spyre AI Accelerator model serving support on x86 platforms

Model serving with the IBM Spyre AI Accelerator is now available as a Technology Preview feature for x86 platforms. The IBM Spyre Operator automates installation and integrates the device plugin, secondary scheduler, and monitoring. For more information, see the [IBM Spyre Operator catalog entry](#).

Build Generative AI Apps with Llama Stack on OpenShift AI

With this release, the Llama Stack Technology Preview feature enables Retrieval-Augmented Generation (RAG) and agentic workflows for building next-generation generative AI applications. It supports remote inference, built-in embeddings, and vector database operations. It also integrates with providers like TrustyAI's provider for safety and Trusty AI's LM-Eval provider for evaluation. This preview includes tools, components, and guidance for enabling the Llama Stack Operator, interacting with the RAG Tool, and automating PDF ingestion and keyword search capabilities to enhance document discovery.

Centralized platform observability

Centralized platform observability, including metrics, traces, and built-in alerts, is available as a Technology Preview feature. This solution introduces a dedicated, pre-configured observability stack for OpenShift AI that allows cluster administrators to perform the following actions:

- View platform metrics (Prometheus) and distributed traces (Tempo) for OpenShift AI components and workloads.
- Manage a set of built-in alerts (alertmanager) that cover critical component health and performance issues.
- Export platform and workload metrics to external 3rd party observability tools by editing the **DataScienceClusterInitialization** (DSCI) custom resource.

You can enable this feature by integrating with the Cluster Observability Operator, Red Hat build of OpenTelemetry, and Tempo Operator. For more information, see [Monitoring and observability](#). For more information, see [Managing observability](#).

Support for Llama Stack Distribution version 0.2.22

The Llama Stack Distribution now includes Llama-stack version 0.2.22 as Technology Preview. This feature brings a number of capabilities, including the following:

- Automatic model registration: Self-hosted model providers, such as vLLM, are now automatically detected and registered in the Llama Stack environment.

- Vector Store API improvements: The Vector Store and Vector Store Files APIs now support more efficient indexing, retrieval, and access control enforcement.
- RAG evaluation providers: Support is now added for pluggable RAG evaluation frameworks, such as Ragas and BEIR, enabling performance and quality benchmarking.
- Access control and telemetry: Expanded role-based access controls for telemetry and data read APIs to improve security and observability.
- Bug fixes and performance updates: Improvements were made to error handling, inference stability, and client initialization across Llama Stack components.

Support for IBM Power accelerated Triton Inference Server

You can now enable Power architecture support for Triton inference server (CPU only) with Python and ONNX backend. You can deploy Triton inference server as a custom model serving runtime on IBM Power architecture as a Technology Preview feature in Red Hat OpenShift AI.

For details, see [Triton Inference Server image](#).

Support for IBM Z accelerated Triton Inference Server

You can now enable Z architecture support for the Triton Inference Server (Telum I/Telum II) with multiple backend options, including ONNX-MLIR, Snap ML (C++), and PyTorch. The Triton Inference Server can be deployed as a custom model serving runtime on IBM Z architecture as a Technology Preview feature in Red Hat OpenShift AI.

For details, see [IBM Z accelerated Triton Inference Server](#).

Support for Kubernetes Event-driven Autoscaling (KEDA)

OpenShift AI now supports Kubernetes Event-driven Autoscaling (KEDA) in its KServe RawDeployment mode. This Technology Preview feature enables metrics-based autoscaling for inference services, allowing for more efficient management of accelerator resources, reduced operational costs, and improved performance for your inference services.

To set up autoscaling for your inference service in KServe RawDeployment mode, you need to install and configure the OpenShift Custom Metrics Autoscaler (CMA), which is based on KEDA.

For more information about this feature, see: [Configuring metrics-based autoscaling](#).

LM-Eval model evaluation UI feature

TrustyAI now offers a user-friendly UI for LM-Eval model evaluations as Technology Preview. This feature allows you to input evaluation parameters for a given model and returns an evaluation-results page, all from the UI.

Use Guardrails Orchestrator with LlamaStack

You can now run detections using the Guardrails Orchestrator tool from TrustyAI with Llama Stack as a Technology Preview feature, using the built-in detection component. To use this feature, ensure TrustyAI is enabled, the FMS Orchestrator and detectors are set up, and KServe RawDeployment mode is in use for full compatibility if needed. There is no manual set up required. Then, in the **DataScienceCluster** custom resource for the Red Hat OpenShift AI Operator, set the **spec.llamastackoperator.managementState** field to **Managed**.

For more information, see [Trusty AI FMS Provider](#) on GitHub.

New Feature Store component

You can now install and manage Feature Store as a configurable component in OpenShift AI. Based on the open-source [Feast](#) project, Feature Store acts as a bridge between ML models and data, enabling consistent and scalable feature management across the ML lifecycle.

This Technology Preview release introduces the following capabilities:

- Centralized feature repository for consistent feature reuse
 - Python SDK and CLI for programmatic and command-line interactions to define, manage, and retrieve features for ML models
 - Feature definition and management
 - Support for a wide range of data sources
 - Data ingestion via feature materialization
 - Feature retrieval for both online model inference and offline model training
 - Role-Based Access Control (RBAC) to protect sensitive features
 - Extensibility and integration with third-party data and compute providers
 - Scalability to meet enterprise ML needs
 - Searchable feature catalog
 - Data lineage tracking for enhanced observability
- For configuration details, see [Configuring Feature Store](#).

IBM Power and IBM Z architecture support

IBM Power (ppc64le) and IBM Z (s390x) architectures are now supported as a Technology Preview feature. Currently, you can only deploy models in **KServe RawDeployment** mode on these architectures.

Support for vLLM in IBM Power and IBM Z architectures

vLLM runtime templates are available for use in IBM Power and IBM Z architectures as Technology Preview.

Enable targeted deployment of workbenches to specific worker nodes in Red Hat OpenShift AI Dashboard using node selectors

Hardware profiles are now available as a Technology Preview. The hardware profiles feature enables users to target specific worker nodes for workbenches or model-serving workloads. It allows users to target specific accelerator types or CPU-only nodes.

This feature replaces the current accelerator profiles feature and container size selector field, offering a broader set of capabilities for targeting different hardware configurations. While accelerator profiles, taints, and tolerations provide some capabilities for matching workloads to hardware, they do not ensure that workloads land on specific nodes, especially if some nodes lack the appropriate taints.

The hardware profiles feature supports both accelerator and CPU-only configurations, along with node selectors, to enhance targeting capabilities for specific worker nodes. Administrators can configure hardware profiles in the settings menu. Users can select the enabled profiles using the UI for workbenches, model serving, and Data Science Pipelines where applicable.

RStudio Server workbench image

With the RStudio Server workbench image, you can access the RStudio IDE, an integrated development environment for R. The R programming language is used for statistical computing and graphics to support data analysis and predictions.

To use the RStudio Server workbench image, you must first build it by creating a secret and triggering the **BuildConfig**, and then enable it in the OpenShift AI UI by editing the **rstudio-rhel9** image stream. For more information, see [Building the RStudio Server workbench images](#).



IMPORTANT

Disclaimer: Red Hat supports managing workbenches in OpenShift AI. However, Red Hat does not provide support for the RStudio software. RStudio Server is available through rstudio.org and is subject to their licensing terms. You should review their licensing terms before you use this sample workbench.

CUDA - RStudio Server workbench image

With the CUDA - RStudio Server workbench image, you can access the RStudio IDE and NVIDIA CUDA Toolkit. The RStudio IDE is an integrated development environment for the R programming language for statistical computing and graphics. With the NVIDIA CUDA toolkit, you can enhance your work by using GPU-accelerated libraries and optimization tools.

To use the CUDA - RStudio Server workbench image, you must first build it by creating a secret and triggering the **BuildConfig**, and then enable it in the OpenShift AI UI by editing the **rstudio-rhel9** image stream. For more information, see [Building the RStudio Server workbench images](#).



IMPORTANT

Disclaimer: Red Hat supports managing workbenches in OpenShift AI. However, Red Hat does not provide support for the RStudio software. RStudio Server is available through rstudio.org and is subject to their licensing terms. You should review their licensing terms before you use this sample workbench.

The CUDA - RStudio Server workbench image contains NVIDIA CUDA technology. CUDA licensing information is available in the [CUDA Toolkit](#) documentation. You should review their licensing terms before you use this sample workbench.

Support for multinode deployment of very large models

Serving models over multiple graphical processing unit (GPU) nodes when using a single-model serving runtime is now available as a Technology Preview feature. Deploy your models across multiple GPU nodes to improve efficiency when deploying large models such as large language models (LLMs). For more information, see [Deploying models by using multiple GPU nodes](#).

CHAPTER 4. DEVELOPER PREVIEW FEATURES



IMPORTANT

This section describes Developer Preview features in Red Hat OpenShift AI 2.25. Developer Preview features are not supported by Red Hat in any way and are not functionally complete or production-ready. Do not use Developer Preview features for production or business-critical workloads. Developer Preview features provide early access to functionality in advance of possible inclusion in a Red Hat product offering. Customers can use these features to test functionality and provide feedback during the development process. Developer Preview features might not have any documentation, are subject to change or removal at any time, and have received limited testing. Red Hat might provide ways to submit feedback on Developer Preview features without an associated SLA.

For more information about the support scope of Red Hat Developer Preview features, see [Developer Preview Support Scope](#).

Distributed Inference with llm-d

Distributed Inference with llm-d is currently available as a Developer Preview feature. Distributed Inference with llm-d supports multi-model serving, intelligent inference scheduling, and disaggregated serving for improved GPU utilization on gen AI models. For more information, see [Deploying models by using Distributed Inference with llm-d](#).

Support for AppWrapper in Kueue

AppWrapper support in Kueue is available as a Developer Preview feature. The experimental API enables the use of AppWrapper-based workloads with the distributed workloads feature.

CHAPTER 5. SUPPORT REMOVALS

This section describes major changes in support for user-facing features in Red Hat OpenShift AI. For information about OpenShift AI supported software platforms, components, and dependencies, see the [Red Hat OpenShift AI: Supported Configurations](#) Knowledgebase article.

5.1. DEPRECATED

5.1.1. Caikit Standalone ServingRuntime for KServe

Starting with OpenShift AI 2.25, the **Caikit Standalone ServingRuntime** for KServe is deprecated and will be removed in a future release of OpenShift AI.

5.1.2. Deprecated Kubeflow Training operator v1

The Kubeflow Training Operator (v1) is deprecated starting OpenShift AI 2.25 and is planned to be removed in a future release. This deprecation is part of our transition to Kubeflow Trainer v2, which delivers enhanced capabilities and improved functionality.

5.1.3. Deprecated TrustyAI service CRD v1alpha1

Starting with OpenShift AI 2.25, the **v1alpha1** version is deprecated and planned for removal in an upcoming release. You must update the TrustyAI Operator to version **v1** to receive future Operator updates.

5.1.4. Deprecated KServe Serverless deployment mode

Starting with OpenShift AI 2.25, The KServe Serverless deployment mode is deprecated. You can continue to deploy models by migrating to the KServe RawDeployment mode. If you are upgrading to Red Hat OpenShift AI 3.0, all workloads that use the retired Serverless or ModelMesh modes must be migrated before upgrading.

5.1.5. Deprecated LAB-tuning

Starting with OpenShift AI 2.25, the LAB-tuning feature is deprecated. If you are using LAB-tuning for large language model customization, plan to migrate to alternative fine-tuning or model customization methods as they become available.

5.1.6. Deprecated embedded Kueue component

Starting with OpenShift AI 2.24, the embedded Kueue component for managing distributed workloads is deprecated. OpenShift AI now uses the Red Hat Build of Kueue Operator to provide enhanced workload scheduling across distributed training, workbench, and model serving workloads. The deprecated embedded Kueue component is not supported in any Extended Update Support (EUS) release. To ensure workloads continue using queue management, you must migrate from the embedded Kueue component to the Red Hat Build of Kueue Operator, which requires OpenShift Container Platform 4.18 or later. To migrate, complete the following steps:

1. Install the Red Hat Build of Kueue Operator from OperatorHub.
2. Edit your **DataScienceCluster** custom resource to set the **spec.components.kueue.managementState** field to **Unmanaged**.

3. Verify that existing Kueue configurations (**ClusterQueue** and **LocalQueue**) are preserved after migration.

For detailed instructions, see [Migrating to the Red Hat build of Kueue Operator](#) .

5.1.7. Deprecated CodeFlare Operator

Starting with OpenShift AI 2.24, the CodeFlare Operator is deprecated and will be removed in a future release of OpenShift AI.



NOTE

This deprecation does not affect the [Red Hat OpenShift AI API tiers](#) .

5.1.8. Deprecated model registry API v1alpha1

Starting with OpenShift AI 2.24, the model registry API version **v1alpha1** is deprecated and will be removed in a future release of OpenShift AI. The latest model registry API version is **v1beta1**.

5.1.9. Multi-model serving platform (ModelMesh)

Starting with OpenShift AI version 2.19, the multi-model serving platform based on ModelMesh is deprecated. You can continue to deploy models on the multi-model serving platform, but it is recommended that you migrate to the single-model serving platform.

For more information or for help on using the single-model serving platform, contact your account manager.

5.1.10. Deprecated Text Generation Inference Server (TGIS)

Starting with OpenShift AI version 2.19, the Text Generation Inference Server (TGIS) is deprecated. TGIS will continue to be supported through the OpenShift AI 2.16 EUS lifecycle. Caikit-TGIS and Caikit are not affected and will continue to be supported. The out-of-the-box serving runtime template will no longer be deployed. vLLM is recommended as a replacement runtime for TGIS.

5.1.11. Deprecated accelerator profiles

Accelerator profiles are now deprecated. To target specific worker nodes for workbenches or model serving workloads, use hardware profiles.

5.1.12. Deprecated OpenVINO Model Server (OVMS) plugin

The CUDA plugin for the OpenVINO Model Server (OVMS) is now deprecated and will no longer be available in future releases of OpenShift AI.

5.1.13. OpenShift AI dashboard user management moved from **OdhDashboardConfig** to **Auth** resource

Previously, cluster administrators used the **groupsConfig** option in the **OdhDashboardConfig** resource to manage the OpenShift groups (both administrators and non-administrators) that can access the OpenShift AI dashboard. Starting with OpenShift AI 2.17, this functionality has moved to the **Auth** resource. If you have workflows (such as GitOps workflows) that interact with **OdhDashboardConfig**, you must update them to reference the **Auth** resource instead.

Table 5.1. Updated configurations

Resource	2.16 and earlier	2.17 and later versions
apiVersion	opendatahub.io/v1alpha	services.platform.opendatahub.io/v1alpha1
kind	OdhDashboardConfig	Auth
name	odh-dashboard-config	auth
Admin groups	spec.groupsConfig.adminGroups	spec.adminGroups
User groups	spec.groupsConfig.allowedGroups	spec.allowedGroups

5.1.14. Deprecated cluster configuration parameters

When using the CodeFlare SDK to run distributed workloads in Red Hat OpenShift AI, the following parameters in the Ray cluster configuration are now deprecated and should be replaced with the new parameters as indicated.

Deprecated parameter	Replaced by
head_cpus	head_cpu_requests, head_cpu_limits
head_memory	head_memory_requests, head_memory_limits
min_cpus	worker_cpu_requests
max_cpus	worker_cpu_limits
min_memory	worker_memory_requests
max_memory	worker_memory_limits
head_gpus	head_extended_resource_requests
num_gpus	worker_extended_resource_requests

You can also use the new **extended_resource_mapping** and **overwrite_default_resource_mapping** parameters, as appropriate. For more information about these new parameters, see the [CodeFlare SDK documentation](#) (external).

5.2. REMOVED FUNCTIONALITY

5.2.1. Microsoft SQL Server command-line tool removal

Starting with OpenShift AI 2.24, the Microsoft SQL Server command-line tools (sqlcmd, bcp) have been removed from workbenches. You can no longer manage Microsoft SQL Server using the preinstalled command-line client.

5.2.2. Model registry ML Metadata (MLMD) server removal

Starting with OpenShift AI 2.23, the ML Metadata (MLMD) server has been removed from the model registry component. The model registry now interacts directly with the underlying database by using the existing model registry API and database schema. This change simplifies the overall architecture and ensures the long-term maintainability and efficiency of the model registry by transitioning from the **ml-metadata** component to direct database access within the model registry itself.

If you see the following error for your model registry deployment, this means that your database schema migration has failed:

```
error: error connecting to datastore: Dirty database version {version}. Fix and force version.
```

You can fix this issue by manually changing the database from a dirty state to 0 before traffic can be routed to the pod. Perform the following steps:

1. Find the name of your model registry database pod as follows:

```
kubectl get pods -n <your-namespace> | grep model-registry-db
```

Replace **<your-namespace>** with the namespace where your model registry is deployed.

2. Use **kubectl exec** to run the query on the model registry database pod as follows:

```
kubectl exec -n <your-namespace> <your-db-pod-name> -c mysql -- mysql -u root -p"$MYSQL_ROOT_PASSWORD" -e "USE <your-db-name>; UPDATE schema_migrations SET dirty = 0;"
```

Replace **<your-namespace>** with your model registry namespace and **<your-db-pod-name>** with the pod name that you found in the previous step. Replace **<your-db-name>** with your model registry database name.

This will reset the dirty state in the database, allowing the model registry to start correctly.

5.2.3. Embedded subscription channel not used in some versions

For OpenShift AI 2.8 to 2.20 and 2.22 to 2.25, the **embedded** subscription channel is not used. You cannot select the **embedded** channel for a new installation of the Operator for those versions. For more information about subscription channels, see [Installing the Red Hat OpenShift AI Operator](#).

5.2.4. Anaconda removal

Anaconda is an open source distribution of the Python and R programming languages. Starting with OpenShift AI version 2.18, Anaconda is no longer included in OpenShift AI, and Anaconda resources are no longer supported or managed by OpenShift AI.

If you previously installed Anaconda from OpenShift AI, a cluster administrator must complete the following steps from the OpenShift command-line interface to remove the Anaconda-related artifacts:

1. Remove the secret that contains your Anaconda password:

oc delete secret -n redhat-ods-applications anaconda-ce-access

2. Remove the **ConfigMap** for the Anaconda validation cronjob:

oc delete configmap -n redhat-ods-applications anaconda-ce-validation-result

3. Remove the Anaconda image stream:

oc delete imagestream -n redhat-ods-applications s2i-minimal-notebook-anaconda

4. Remove the Anaconda job that validated the downloading of images:

oc delete job -n redhat-ods-applications anaconda-ce-periodic-validator-job-custom-run

5. Remove any pods related to Anaconda cronjob runs:

oc get pods -n redhat-ods-applications --no-headers=true | awk '/anaconda-ce-periodic-validator-job-custom-run*/'

5.2.5. Pipeline logs for Python scripts running in Elyra pipelines are no longer stored in S3

Logs are no longer stored in S3-compatible storage for Python scripts which are running in Elyra pipelines. From OpenShift AI version 2.11, you can view these logs in the pipeline log viewer in the OpenShift AI dashboard.



NOTE

For this change to take effect, you must use the Elyra runtime images provided in workbench images at version 2024.1 or later.

If you have an older workbench image version, update the **Version selection** field to a compatible workbench image version, for example, 2024.1, as described in [Updating a project workbench](#).

Updating your workbench image version will clear any existing runtime image selections for your pipeline. After you have updated your workbench version, open your workbench IDE and update the properties of your pipeline to select a runtime image.

5.2.6. Beta subscription channel no longer used

Starting with OpenShift AI 2.5, the **beta** subscription channel is no longer used. You can no longer select the **beta** channel for a new installation of the Operator. For more information about subscription channels, see [Installing the Red Hat OpenShift AI Operator](#).

5.2.7. HabanaAI workbench image removal

Support for the HabanaAI 1.10 workbench image has been removed. New installations of OpenShift AI from version 2.14 do not include the HabanaAI workbench image. However, if you upgrade OpenShift AI from a previous version, the HabanaAI workbench image remains available, and existing HabanaAI workbench images continue to function.

CHAPTER 6. RESOLVED ISSUES

The following notable issues are resolved in Red Hat OpenShift AI 2.25.3. Security updates, bug fixes, and enhancements for Red Hat OpenShift AI 2.25 are released as asynchronous errata. All OpenShift AI errata advisories are published on the [Red Hat Customer Portal](#).

6.1. SECURITY UPDATES IN RED HAT OPENSIFT AI 2.25.3 (FEB 2026)

This release provides security updates. For a complete list of updates, see the associated errata advisory on the [Red Hat Customer Portal](#)

6.2. ISSUES RESOLVED IN RED HAT OPENSIFT AI 2.25.2 (FEBRUARY 2026)

This release provides security updates. For a complete list of updates, see the associated errata advisory on the [Red Hat Customer Portal](#)

RHOAIENG-45145 - OAuth proxy error returning wrong HTTP code

Before this update, OAuth proxy sent incorrect "HTTP 302" responses for unauthorized requests and issued invalid tokens. This issue is now resolved.

RHOAIENG-47370 - KServe Raw Timeout

Before this update, a set value was given for predictor-timeout and was not configurable through the UI. This issue is now resolved.

6.3. SECURITY UPDATES IN RED HAT OPENSIFT AI 2.25.1 (DECEMBER 2025)

This release provides security updates. For a complete list of updates, see the associated errata advisory on the [Red Hat Customer Portal](#).

RHOAIENG-35623 - Model deployment fails when using hardware profiles

Previously, model deployments that used hardware profiles failed because the Red Hat OpenShift AI Operator did not inject the **tolerations**, **nodeSelector**, or **identifiers** from the hardware profile into the underlying **InferenceService** when manually creating **InferenceService** resources. As a result, the model deployment pods could not be scheduled to suitable nodes and the deployment fails to enter a ready state. This issue is now resolved.

RHOAIENG-35532 - Unable to deploy models with HardwareProfiles and GPU

Before this update, the HardwareProfile to use GPU for model deployment had stopped working. The issue is now resolved.

RHOAIENG-34528 - Requests time out after 30 seconds in KServe RawDeployment mode

Before this update, when you used KServe RawDeployment mode in OpenShift AI with an OAuth proxy, a default 30-second timeout applied to requests. As a result, any request that took longer than 30 seconds failed before completion. This issue is now resolved.

RHOAIENG-33914 - LM-Eval Tier2 task test failures

Previously, there could be failures with LM-Eval Tier2 task tests because the Massive Multitask Language Understanding Symbol Replacement (MMLUSR) tasks were broken. This issue is resolved with the latest version of the **trustyai-service-operator**.

RHOAIENG-32942 - Elyra requires unsupported filters on the REST API when pipeline store is Kubernetes

Before this update, when the pipeline store was configured to use Kubernetes, Elyra required equality (**eq**) filters that were not supported by the REST API. Only substring filters were supported in this mode. As a result, pipelines created and submitted through Elyra from a workbench could not run successfully. This issue is now resolved.

RHOAIENG-32897 - Pipelines defined with the Kubernetes API and invalid `platformSpec` do not appear in the UI or run

Before this update, when a pipeline version defined with the Kubernetes API included an empty or invalid **`spec.platformSpec`** field (for example, `{}` or missing the **`kubernetes`** key), the system misidentified the field as the pipeline specification. As a result, the REST API omitted the **`pipelineSpec`**, which prevented the pipeline version from being displayed in the UI and from running. This issue is now resolved.

RHOAIENG-31386 - Error deploying an Inference Service with `authenticationRef`

Before this update, when deploying an InferenceService with **`authenticationRef`** under external metrics, the **`authenticationRef`** field was removed. This issue is now resolved.

RHOAIENG-30493 - Error creating a workbench in a Kueue-enabled project

Previously, when using the dashboard to create a workbench in a Kueue-enabled project, the creation failed if Kueue was disabled on the cluster or if the selected hardware profile was not associated with a LocalQueue. In this case, the required LocalQueue could not be referenced, the admission webhook validation failed, and an error message was shown. This issue is now resolved.

RHOAIENG-9418 - Elyra raises error when you use parameters in uppercase

Previously, Elyra raised an error when you tried to run a pipeline that used parameters in uppercase. This issue is now resolved.

RHOAIENG-4570 - Existing Argo Workflows installation conflicts with install or upgrade

Previously, installing or upgrading OpenShift AI on a cluster that already included an existing Argo Workflows instance could cause conflicts with the embedded Argo components deployed by Data Science Pipelines. This issue has been resolved. You can now configure OpenShift AI to use an existing Argo Workflows instance, enabling clusters that already run Argo Workflows to integrate with Data Science Pipelines without conflicts.

CHAPTER 7. KNOWN ISSUES

This section describes known issues in Red Hat OpenShift AI 2.25.3 and any known methods of working around these issues.

RHOAIENG-46747 - NetworkPolicy blocks OpenShift Route traffic

By default, LlamaStack servers are protected by network policies and only allows traffic within the **LlamaStackDistribution** namespace. Workaround:: To enable traffic from other OpenShift namespaces, add the following network configurations to your **LlamaStackDistribution** custom resource file.

```
spec:
  network:
    allowedFrom:
      namespaces:
        - <namespace>
      labels:
        - <project-name>/lls-allowed
```

Then, to enable OpenShift routes to you need to add the following network policy to your **LlamaStackDistribution** custom resource file.

```
spec:
  ingress:
    - ports:
        - protocol: TCP
          port: 8321
      from:
        - namespaceSelector:      matchLabels:
            policy-group.network.openshift.io/ingress: "
        - namespaceSelector:
            matchLabels:
              policy-group.network.openshift.io/host-network: "
```

RHAIENG-1139 - Cannot deploy LlamaStackDistribution with the same name in multiple namespaces

If you create two **LlamaStackDistribution** resources with the same name in different namespaces, the ReplicaSet for the second resource fails to start the Llama Stack pod. The Llama Stack Operator does not correctly assign security constraints when duplicate names are used across namespaces.

Workaround

Use a unique name for each **LlamaStackDistribution** in every namespace. For example, include the project name or add a suffix such as **llama-stack-distribution-209342**.

RHAIENG-1624 - Embeddings API timeout on disconnected clusters

On disconnected clusters, calls to the embeddings API might time out when using the default embedding model (**ibm-granite/granite-embedding-125m-english**) included in the default Llama Stack distribution image.

Workaround

Add the following environment variables to the **LlamaStackDistribution** custom resource to use the embedded model offline:

```
- name: SENTENCE_TRANSFORMERS_HOME
  value: /opt/app-root/src/.cache/huggingface/hub
- name: HF_HUB_OFFLINE
  value: "1"
- name: TRANSFORMERS_OFFLINE
  value: "1"
- name: HF_DATASETS_OFFLINE
  value: "1"
```

RHOAIENG-34923 - Runtime configuration missing when running a pipeline from JupyterLab

The runtime configuration might not appear in the Elyra pipeline editor when you run a pipeline from the first active workbench in a project. This occurs because the configuration fails to populate for the initial workbench session.

Workaround

Restart the workbench. After restarting, the runtime configuration becomes available for pipeline execution.

RHAIENG-35055 - Model catalog fails to initialize after upgrading from OpenShift AI 2.24

After upgrading from OpenShift AI 2.24, the model catalog might fail to initialize and load. The OpenShift AI dashboard displays a **Request access to model catalog** error.

Workaround

Delete the existing model catalog ConfigMap and deployment by running the following commands:

```
$ oc delete configmap model-catalog-sources -n rhoai-model-registries --ignore-not-found
$ oc delete deployment model-catalog -n rhoai-model-registries --ignore-not-found
```

RHAIENG-35529 - Reconciliation issues in Data Science Pipelines Operator when using external Argo Workflows

If you enable the embedded Argo Workflows controllers (**argoWorkflowsControllers: Managed**) before deleting an existing external Argo Workflows installation, the workflow controller might fail to start and the Data Science Pipelines Operator (DSPO) might not reconcile its custom resources correctly.

Workaround

Before enabling the embedded Argo Workflows controllers, delete any existing external Argo Workflows instance from the cluster.

RHAIENG-36756 - Existing cluster storage option missing during model deployment when no connections exist

When creating a model deployment in a project with no defined data connections, the **Existing cluster storage** option does not appear, even if Persistent Volume Claims (PVCs) are available. As a result, you cannot select an existing PVC for model storage.

Workaround

Create at least one connection of type **URI** in the project. Afterward, the **Existing cluster storage** option becomes available.

RHOAIENG-36817 - Inference server fails when Model server size is set to **small**

When creating an inference service via the dashboard, selecting a **small** Model server size causes subsequent inferencing requests to fail. As a result, the deployment of the inference service itself succeeds, but the inferencing requests fail with a timeout error.

Workaround

To resolve this issue, select the Model server size as **large** from the dropdown.

RHOAIENG-33995 - Deployment of an inference service for Phi and Mistral models fails

The creation of an inference service for Phi and Mistral models using vLLM runtime on IBM Power cluster with openshift-container-platform 4.19 fails due to an error related to CPU backend. As a result, deployment of these models is affected, causing inference service creation failure.

Workaround

To resolve this issue, disable the **sliding_window** mechanism in the serving runtime if it is enabled for CPU and Phi models. Sliding window is not currently supported in V1.

RHOAIENG-33795 - Manual **Route** creation needed for gRPC endpoint verification for Triton Inference Server on IBM Z

When verifying Triton Inference Server with gRPC endpoint, **Route** does not get created automatically. This happens because the Operator currently defaults to creating an edge-terminated route for REST only.

Workaround

To resolve this issue, manual Route creation is needed for gRPC endpoint verification for Triton Inference Server on IBM Z.

1. When the model deployment pod is up and running, define an edge-terminated **Route** object in a YAML file with the following contents:

```
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: <grpc-route-name>           # e.g. triton-grpc
  namespace: <model-deployment-namespace> # namespace where your model is
  deployed
labels:
  inferencesservice-name: <inference-service-name>
annotations:
  haproxy.router.openshift.io/timeout: 30s
spec:
  host: <custom-hostname>           # e.g. triton-grpc.<apps-domain>
  to:
    kind: Service
    name: <service-name>             # name of the predictor service (e.g. triton-predictor)
    weight: 100
  port:
    targetPort: grpc                 # must match the gRPC port exposed by the service
```

```
tls:
  termination: edge
  wildcardPolicy: None
```

2. Create the **Route** object:

```
oc apply -f <route-file-name>.yaml
```

3. To send an inference request, enter the following command:

```
grpcurl -cacert <ca_cert_file>\ 1
  -protoset triton_desc.pb \
  -d '{
    "model_name": "<model_name>",
    "inputs": [
      {
        "name": "<input_tensor_name>",
        "shape": [<shape>],
        "datatype": "<data_type>",
        "contents": {
          "<datatype_specific_contents>": [<input_data_values>]
        }
      }
    ],
    "outputs": [
      {
        "name": "<output_tensor_name>"
      }
    ]
  }' \
  <grpc_route_host>:443 \
  inference.GRPCInferenceService/ModelInfer
```

1 <ca_cert_file> is the path to your cluster router CA cert (for example, router-ca.crt).



NOTE

<triton_protoset_file> is compiled as a protobuf descriptor file. You can generate it as **protoc -I. --descriptor_set_out=triton_desc.pb --include_imports grpc_service.proto**.

Download **grpc_service.proto** and **model_config.proto** files from the [triton-inference-service](#) GitHub page.

RHOAIENG-33697 - Unable to Edit or Delete models unless status is "Started"

When you deploy a model on the NVIDIA NIM or single-model serving platform, the **Edit** and **Delete** options in the action menu are not available for models in the **Starting** or **Pending** states. These options become available only after the model has been successfully deployed.

Workaround

Wait until the model is in the **Started** state to make any changes or to delete the model.

RHOAIENG-33645 - LM-Eval Tier1 test failures

There can be failures with LM-Eval Tier1 tests because **confirm_run_unsafe_code** is not passed as an argument when a job is run, if you are using an older version of the **trustyai-service-operator**.

Workaround

Ensure that you are using the latest version of the **trustyai-service-operator** and that **AllowCodeExecution** is enabled.

RHOAIENG-29729 - Model registry Operator in a restart loop after upgrade

After upgrading from OpenShift AI version 2.22 or earlier to version 2.23 or later with the model registry component enabled, the model registry Operator might enter a restart loop. This is due to an insufficient memory limit for the manager container in the **model-registry-operator-controller-manager** pod.

Workaround

To resolve this issue, you must trigger a reconciliation for the **model-registry-operator-controller-manager** deployment. Adding the **opendatahub.io/managed='true'** annotation to the deployment will accomplish this and apply the correct memory limit. You can add the annotation by running the following command:

```
oc annotate deployment model-registry-operator-controller-manager -n redhat-ods-applications
opendatahub.io/managed='true' --overwrite
```

**NOTE**

This command overwrites custom values in the **model-registry-operator-controller-manager** deployment. For more information about custom deployment values, see [Customizing component deployment resources](#).

After the deployment updates and the memory limit increases from 128Mi to 256Mi, the container memory usage will stabilize and the restart loop will stop.

RHOAIENG-31238 - New observability stack enabled when creating DSCInitialization

When you remove a DSCInitialization resource and create a new one using OpenShift AI console form view, it enables a Technology Preview observability stack. This results in the deployment of an unwanted observability stack when recreating a DSCInitialization resource.

Workaround

To resolve this issue, manually remove the "metrics" and "traces" fields when recreating the DSCInitialization resource using the form view.

This is not required if you want to use the Technology Preview observability stack.

RHOAIENG-32145 - Llama Stack Operator deployment failures on OpenShift versions earlier than 4.17

When installing OpenShift AI on OpenShift clusters running versions earlier than 4.17, the integrated Llama Stack Operator (**llamastackoperator**) might fail to deploy.

The Llama Stack Operator requires Kubernetes version 1.32 or later, but OpenShift 4.15 uses Kubernetes 1.28. This version gap can cause schema validation failures when applying the **LlamaStackDistribution** custom resource definition (CRD), due to unsupported selectable fields introduced in Kubernetes 1.32.

Workaround

Install OpenShift AI on an OpenShift cluster running version 4.17 or later.

[RHOAIENG-32242](#) - Failure on creating NetworkPolicies for OpenShift versions 4.15 and 4.16

When installing OpenShift AI on OpenShift clusters running versions 4.15 or 4.16, deployment of certain **NetworkPolicy** resources might fail. This can occur when the **llamastackoperator** or related components attempt to create a **NetworkPolicy** in a protected namespace, such as **redhat-ods-applications**. The request can be blocked by the admission webhook **networkpolicies-validation.managed.openshift.io**, which restricts modifications to certain namespaces and resources, even for **cluster-admin** users. This restriction can apply to both self-managed and Red Hat-managed OpenShift environments.

Workaround

Deploy OpenShift AI on an OpenShift cluster running version 4.17 or later. For clusters where the webhook restriction is enforced, contact your OpenShift administrator or Red Hat Support to determine an alternative deployment pattern or approved change to the affected namespace.

[RHOAIENG-32599](#) - Inference service creation fails on IBM Z cluster

When you attempt to create an inference service using the vLLM runtime on an IBM Z cluster, it fails with the following error: **ValueError: 'aimv2' is already used by a Transformers config, pick another name**.

Workaround

None.

[RHOAIENG-29731](#) - Inference service creation fails on IBM Power cluster with OpenShift 4.19

When you attempt to create an inference service by using the vLLM runtime on an IBM Power cluster on OpenShift Container Platform version 4.19, it fails due to an error related to Non-Uniform Memory Access (NUMA).

Workaround

When you create an inference service, set the environment variable **VLLM_CPU_OMP_THREADS_BIND** to **all**.

[RHOAIENG-29292](#) - vLLM logs permission errors on IBM Z due to usage stats directory access

When running vLLM on the IBM Z architecture, the inference service starts successfully, but logs an error in a background thread related to usage statistics reporting. This happens because the service tries to write usage data to a restricted location (**/.config**), which it does not have permission to access.

The following error appears in the logs:

```
Exception in thread Thread-2 (_report_usage_worker):
Traceback (most recent call last):
...
PermissionError: [Error 13] Permission denied: '/.config'
```

Workaround

To prevent this error and suppress the usage stats logging, set the **VLLM_NO_USAGE_STATS=1** environment variable in the inference service deployment. This disables automatic usage reporting, avoiding permission issues when you write to system directories.

RHOAIENG-28910 - Unmanaged KServe resources are deleted after upgrading from 2.16 to 2.19 or later

During the upgrade from OpenShift AI 2.16 to 2.25, the **FeatureTracker** custom resource (CR) is deleted before its owner references are fully removed from associated KServe-related resources. As a result, resources that were originally created by the Red Hat OpenShift AI Operator with a **Managed** state and later changed to **Unmanaged** in the **DataScienceCluster** (DSC) custom resource (CR) might be unintentionally removed. This issue can disrupt model serving functionality until the resources are manually restored.

The following resources might be deleted in 2.25 if they were changed to **Unmanaged** in 2.16:

Kind	Namespace	Name
KnativeServing	knative-serving	knative-serving
ServiceMeshMember	knative-serving	default
Gateway	istio-system	kserve-local-gateway
Gateway	knative-serving	knative-ingress-gateway
Gateway	knative-serving	knative-local-gateway

Workaround

If you have already upgraded from OpenShift AI 2.16 to 2.25, perform one of the following actions:

- If you have an existing backup, manually recreate the deleted resources without owner references to the **FeatureTracker** CR.
- If you do not have an existing backup, you can use the Operator to recreate the deleted resources:
 1. Back up any resources you have already recreated.
 2. In the DSC, set **spec.components.kserve.serving.managementState** to **Managed**, and then save the change to allow the Operator to recreate the resources. Wait until the Operator has recreated the resources.
 3. In the DSC, set **spec.components.kserve.serving.managementState** back to **Unmanaged**, and then save the change.
 4. Reapply any previous custom changes to the recreated **KnativeServing**, **ServiceMeshMember**, and **Gateway** CRs resources.

If you have not yet upgraded, perform the following actions before upgrading to prevent this issue:

1. In the DSC, set **spec.components.kserve.serving.managementState** to **Unmanaged**.

- For each of the affected **KnativeServing**, **ServiceMeshMember**, and **Gateway** resources listed in the above table, edit its CR by deleting the **FeatureTracker** owner reference. This edit removes the resource's dependency on the **FeatureTracker** and prevents the deletion of the resource during the upgrade process.

RHOAIENG-24545 - Runtime images are not present in the workbench after the first start

The list of runtime images does not properly populate the first running workbench instance in the namespace, therefore no image is shown for selection in the Elyra pipeline editor.

Workaround

Restart the workbench. After restarting the workbench, the list of runtime images populates both the workbench and the select box for the Elyra pipeline editor.

RHOAIENG-24786 - Upgrading the Authorino Operator from Technical Preview to Stable fails in disconnected environments

In disconnected environments, upgrading the Red Hat Authorino Operator from Technical Preview to Stable fails with an error in the **authconfig-migrator-qqtz** pod.

Workaround

- Update the Red Hat Authorino Operator to the latest version in the **tech-preview-v1** update channel (v1.1.2).
- Run the following script:

```
#!/bin/bash
set -xe

# delete the migrator job
oc delete job -n openshift-operators authconfig-migrator || true

# run the migrator script
authconfigs=$(oc get authconfigs -A -o custom-
columns='NAMESPACE:.metadata.namespace,NAME:.metadata.name' --no-headers)

if [[ ! -z "${authconfigs}" ]]; then
  while IFS=" " read -r namespace name; do
    oc get authconfig "$name" -n "$namespace" -o yaml >
"/tmp/${name}.${namespace}.authconfig.yaml"
    oc apply -f "/tmp/${name}.${namespace}.authconfig.yaml"
  done <<< "${authconfigs}"
fi

oc patch crds authconfigs.authorino.kuadrant.io --type=merge --subresource status --
patch '{"status":{"storedVersions":["v1beta2"]}]'
```

- Update the Red Hat Authorino Operator subscription to use the **stable** update channel.
- Select the update option for Authorino 1.2.1.

RHOAIENG-20209 - Warning message not displayed when requested resources exceed threshold

When you click **Distributed workloads** → **Project metrics** and view the **Requested resources** section, the charts show the requested resource values and the total shared quota value for each resource (**CPU** and **Memory**). However, when the **Requested by all projects** value exceeds the **Warning threshold** value for that resource, the expected warning message is not displayed.

Workaround

None.

[SRVKS-1301](#) (previously documented as [RHOAIENG-18590](#)) - The **KnativeServing** resource fails after disabling and enabling KServe

After disabling and enabling the **kserve** component in the **DataScienceCluster**, the **KnativeServing** resource might fail.

Workaround

Delete all **ValidatingWebhookConfiguration** and **MutatingWebhookConfiguration** webhooks related to Knative:

1. Get the webhooks:

```
oc get ValidatingWebhookConfiguration,MutatingWebhookConfiguration | grep -i knative
```

2. Ensure KServe is disabled.

3. Get the webhooks:

```
oc get ValidatingWebhookConfiguration,MutatingWebhookConfiguration | grep -i knative
```

4. Delete the webhooks.

5. Enable KServe.

6. Verify that the KServe pod can successfully spawn, and that pods in the **knative-serving** namespace are active and operational.

[RHOAIENG-16247](#) - Elyra pipeline run outputs are overwritten when runs are launched from OpenShift AI dashboard

When a pipeline is created and run from Elyra, outputs generated by the pipeline run are stored in the folder **bucket-name/pipeline-name-timestamp** of object storage.

When a pipeline is created from Elyra and the pipeline run is started from the OpenShift AI dashboard, the timestamp value is not updated. This can cause pipeline runs to overwrite files created by previous pipeline runs of the same pipeline.

This issue does not affect pipelines compiled and imported using the OpenShift AI dashboard because **runid** is always added to the folder used in object storage. For more information about storage locations used in data science pipelines, see [Storing data with data science pipelines](#).

Workaround

When storing files in an Elyra pipeline, use different subfolder names on each pipeline run.

[OCPBUGS-49422](#) - AMD GPUs and AMD ROCm workbench images are not supported in a disconnected environment

This release of OpenShift AI does not support AMD GPUs and AMD ROCm workbench images in a disconnected environment because installing the AMD GPU Operator requires internet access to fetch dependencies needed to compile GPU drivers.

Workaround

None.

[RHOAIENG-12516](#) - **fast** releases are available in unintended release channels

Due to a known issue with the stream image delivery process, **fast** releases are currently available on unintended streaming channels, for example, **stable**, and **stable-x.y**. For accurate release type, channel, and support lifecycle information, refer to the **Life-cycle Dates** table on the [Red Hat OpenShift AI Self-Managed Life Cycle](#) page.

Workaround

None.

[RHOAIENG-8294](#) - CodeFlare error when upgrading OpenShift AI 2.8 to version 2.10 or later

If you try to upgrade OpenShift AI 2.8 to version 2.10 or later, the following error message is shown for the CodeFlare component, due to a mismatch with the **AppWrapper** custom resource definition (CRD) version.

```
ReconcileCompletedWithComponentErrors DataScienceCluster resource reconciled with component errors: 1 error occurred: * CustomResourceDefinition.apiextensions.k8s.io "appwrappers.workload.codeflare.dev" is invalid: status.storedVersions[0]: Invalid value: "v1beta1": must appear in spec.versions
```

Workaround

1. Delete the existing **AppWrapper** CRD:

```
$ oc delete crd appwrappers.workload.codeflare.dev
```

2. Wait for about 20 seconds, and then ensure that a new **AppWrapper** CRD is automatically applied, as shown in the following example:

```
$ oc get crd appwrappers.workload.codeflare.dev
NAME                                CREATED AT
appwrappers.workload.codeflare.dev  2024-11-22T18:35:04Z
```

[RHOAIENG-7716](#) - Pipeline condition group status does not update

When you run a pipeline that has loops (**dsl.ParallelFor**) or condition groups (**dsl.If**), the UI displays a Running status for the loops and groups, even after the pipeline execution is complete.

Workaround

You can confirm if a pipeline is still running by checking that no child tasks remain active.

1. From the OpenShift AI dashboard, click **Data Science Pipelines → Runs**.
2. From the **Project** list, click your data science project.

3. From the **Runs** tab, click the pipeline run that you want to check the status of.
4. Expand the condition group and click a child task.
A panel that contains information about the child task is displayed
5. On the panel, click the **Task** details tab.
The **Status** field displays the correct status for the child task.

RHOAIENG-6409 - Cannot save parameter errors appear in pipeline logs for successful runs

When you run a pipeline more than once with data science pipelines 2.0, **Cannot save parameter** errors appear in the pipeline logs for successful pipeline runs. You can safely ignore these errors.

Workaround

None.

RHOAIENG-3025 - OVMS expected directory layout conflicts with the KServe StoragePuller layout

When you use the OpenVINO Model Server (OVMS) runtime to deploy a model on the single-model serving platform (which uses KServe), there is a mismatch between the directory layout expected by OVMS and that of the model-pulling logic used by KServe. Specifically, OVMS requires the model files to be in the `/<mnt>/models/1/` directory, while KServe places them in the `/<mnt>/models/` directory.

Workaround

Perform the following actions:

1. In your S3-compatible storage bucket, place your model files in a directory called **1/**, for example, `/<s3_storage_bucket>/models/1/<model_files>`.
2. To use the OVMS runtime to deploy a model on the single-model serving platform, choose one of the following options to specify the path to your model files:
 - If you are using the OpenShift AI dashboard to deploy your model, in the **Path** field for your data connection, use the `/<s3_storage_bucket>/models/` format to specify the path to your model files. Do not specify the **1/** directory as part of the path.
 - If you are creating your own **InferenceService** custom resource to deploy your model, configure the value of the **storageURI** field as `/<s3_storage_bucket>/models/`. Do not specify the **1/** directory as part of the path.

KServe pulls model files from the subdirectory in the path that you specified. In this case, KServe correctly pulls model files from the `/<s3_storage_bucket>/models/1/` directory in your S3-compatible storage.

RHOAIENG-3018 - OVMS on KServe does not expose the correct endpoint in the dashboard

When you use the OpenVINO Model Server (OVMS) runtime to deploy a model on the single-model serving platform, the URL shown in the **Inference endpoint** field for the deployed model is not complete.

Workaround

To send queries to the model, you must add the `/v2/models/_<model-name>_/infer` string to the end of the URL. Replace `_<model-name>_` with the name of your deployed model.

RHOAIENG-2602 - "Average response time" server metric graph shows multiple lines due to ModelMesh pod restart

The **Average response time** server metric graph shows multiple lines if the ModelMesh pod is restarted.

Workaround

None.

RHOAIENG-2228 - The performance metrics graph changes constantly when the interval is set to 15 seconds

On the **Endpoint performance** tab of the model metrics screen, if you set the **Refresh interval** to 15 seconds and the **Time range** to 1 hour, the graph results change continuously.

Workaround

None.

RHOAIENG-2183 - Endpoint performance graphs might show incorrect labels

In the **Endpoint performance** tab of the model metrics screen, the graph tooltip might show incorrect labels.

Workaround

None.

RHOAIENG-131 - gRPC endpoint not responding properly after the InferenceService reports as Loaded

When numerous **InferenceService** instances are generated and directed requests, Service Mesh Control Plane (SMCP) becomes unresponsive. The status of the **InferenceService** instance is **Loaded**, but the call to the gRPC endpoint returns with errors.

Workaround

Edit the **ServiceMeshControlPlane** custom resource (CR) to increase the memory limit of the Istio egress and ingress pods.

RHOAIENG-1619 (previously documented as DATA-SCIENCE-PIPELINES-165) - Poor error message when S3 bucket is not writable

When you set up a data connection and the S3 bucket is not writable, and you try to upload a pipeline, the error message **Failed to store pipelines** is not helpful.

Workaround

Verify that your data connection credentials are correct and that you have write access to the bucket you specified.

RHOAIENG-1207 (previously documented as ODH-DASHBOARD-1758) - Error duplicating OOTB custom serving runtimes several times

If you duplicate a model-serving runtime several times, the duplication fails with the **Serving runtime name "<name>" already exists** error message.

Workaround

Change the **metadata.name** field to a unique value.

RHOAIENG-133 - Existing workbench cannot run Elyra pipeline after workbench restart

If you use the Elyra JupyterLab extension to create and run data science pipelines within JupyterLab, and you configure the pipeline server *after* you created a workbench and specified a workbench image within the workbench, you cannot execute the pipeline, even after restarting the workbench.

Workaround

1. Stop the running workbench.
2. Edit the workbench to make a small modification. For example, add a new dummy environment variable, or delete an existing unnecessary environment variable. Save your changes.
3. Restart the workbench.
4. In the left sidebar of JupyterLab, click **Runtimes**.
5. Confirm that the default runtime is selected.

RHODS-12798 - Pods fail with "unable to init seccomp" error

Pods fail with **CreateContainerError** status or **Pending** status instead of **Running** status, because of a known kernel bug that introduced a **seccomp** memory leak. When you check the events on the namespace where the pod is failing, or run the **oc describe pod** command, the following error appears:

```
runc create failed: unable to start container process: unable to init seccomp: error loading seccomp filter into kernel: error loading seccomp filter: errno 524
```

Workaround

Increase the value of **net.core.bpf_jit_limit** as described in the Red Hat Knowledgebase solution [Pods failing with error loading seccomp filter into kernel: errno 524 in OpenShift 4](#).

KUBEFLOW-177 - Bearer token from application not forwarded by OAuth-proxy

You cannot use an application as a custom workbench image if its internal authentication mechanism is based on a bearer token. The OAuth-proxy configuration removes the bearer token from the headers, and the application cannot work properly.

Workaround

None.

KUBEFLOW-157 - Logging out of JupyterLab does not work if you are already logged out of the OpenShift AI dashboard

If you log out of the OpenShift AI dashboard before you log out of JupyterLab, then logging out of JupyterLab is not successful. For example, if you know the URL for a Jupyter notebook, you are able to open this again in your browser.

Workaround

Log out of JupyterLab before you log out of the OpenShift AI dashboard.

RHODS-7718 - User without dashboard permissions is able to continue using their running workbenches indefinitely

When a Red Hat OpenShift AI administrator revokes a user's permissions, the user can continue to use their running workbenches indefinitely.

Workaround

When the OpenShift AI administrator revokes a user's permissions, the administrator should also stop any running workbenches for that user.

RHOAIENG-1152 (previously documented as **RHODS-6356**) - The basic-workbench creation process fails for users who have never logged in to the dashboard

The dashboard's **Administration** page for basic workbenches displays users who belong to the user group and admin group in OpenShift. However, if an administrator attempts to start a basic workbench on behalf of a user who has never logged in to the dashboard, the basic-workbench creation process fails and displays the following error message:

```
Request invalid against a username that does not exist.
```

Workaround

Request that the relevant user logs into the dashboard.

RHODS-5543 - When using the NVIDIA GPU Operator, more nodes than needed are created by the Node Autoscaler

When a pod cannot be scheduled due to insufficient available resources, the Node Autoscaler creates a new node. There is a delay until the newly created node receives the relevant GPU workload. Consequently, the pod cannot be scheduled and the Node Autoscaler's continuously creates additional new nodes until one of the nodes is ready to receive the GPU workload. For more information about this issue, see the Red Hat Knowledgebase solution [When using the NVIDIA GPU Operator, more nodes than needed are created by the Node Autoscaler](#).

Workaround

Apply the **cluster-api/accelerator** label in **machineset.spec.template.spec.metadata**. This causes the autoscaler to consider those nodes as unready until the GPU driver has been deployed.

RHODS-4799 - Tensorboard requires manual steps to view

When a user has TensorFlow or PyTorch workbench images and wants to use TensorBoard to display data, manual steps are necessary to include environment variables in the workbench environment, and to import those variables for use in your code.

Workaround

When you start your basic workbench, use the following code to set the value for the `TENSORBOARD_PROXY_URL` environment variable to use your OpenShift AI user ID.

```
import os
os.environ["TENSORBOARD_PROXY_URL"] = os.environ["NB_PREFIX"] + "/proxy/6006/"
```

RHODS-4718 - The Intel® oneAPI AI Analytics Toolkits quick start references nonexistent sample notebooks

The Intel® oneAPI AI Analytics Toolkits quick start, located on the **Resources** page on the dashboard, requires the user to load sample notebooks as part of the instruction steps, but refers to notebooks that do not exist in the associated repository.

Workaround

None.

RHODS-3984 - Incorrect package versions displayed during notebook selection

In the OpenShift AI interface, the **Start a notebook server page** displays incorrect version numbers for the JupyterLab and Notebook packages included in the oneAPI AI Analytics Toolkit notebook image. The page might also show an incorrect value for the Python version used by this image.

Workaround

When you start your oneAPI AI Analytics Toolkit notebook server, you can check which Python packages are installed on your notebook server and which version of the package you have by running the **!pip list** command in a notebook cell.

RHOAING-1147 (previously documented as RHODS-2881) - Actions on dashboard not clearly visible

The dashboard actions to revalidate a disabled application license and to remove a disabled application tile are not clearly visible to the user. These actions appear when the user clicks on the application tile's **Disabled** label. As a result, the intended workflows might not be clear to the user.

Workaround

None.

RHODS-2096 - IBM Watson Studio not available in OpenShift AI

IBM Watson Studio is not available when OpenShift AI is installed on OpenShift Dedicated 4.9 or higher, because it is not compatible with these versions of OpenShift Dedicated.

Workaround

Contact the [Red Hat Customer Portal](#) for assistance with manually configuring Watson Studio on OpenShift Dedicated 4.9 and higher.

CHAPTER 8. PRODUCT FEATURES

Red Hat OpenShift AI provides a rich set of features for data scientists and cluster administrators. To learn more, see [Introduction to Red Hat OpenShift AI](#).